Advanced Queries

JOIN, UNION, GROUP BY & HAVING — Subqueries & SQL Execution Order

Fettah KIRAN
Department of AI and Data Engineering
Yıldız Technical University
fkiran@yildiz.edu.tr

📋 Today's Agenda (1:00 PM – 3:45 PM)

Time	Topic	Type
1:00 – 1:10	Week 2 recap quiz	Review
1:10 – 1:50	JOINs: INNER, LEFT, RIGHT, CROSS, Self	**Theory** 📖
1:50 – 2:15	GROUP BY, Aggregate Functions & HAVING	**Theory** 📖
2:15 – 2:30	☕ Break (15 min)	
2:30 – 2:50	UNION, INTERSECT, EXCEPT & Set Operations	**Theory** 📖
2:50 – 3:10	Subqueries & SQL Execution Order	**Theory** 📖
3:10 – 3:40	Hands-on: Multi-table Queries	**Practice** 💻
3:40 – 3:45	Recap & next week preview	Wrap-up

Week 2 Quick Recap

Concept	Key Point
SQLite Types	INTEGER, REAL, TEXT, BLOB, NULL
Type Affinity	Values typed dynamically, columns have affinity
PRIMARY KEY	Unique row identifier, NOT NULL
NOT NULL	Prevents empty values
UNIQUE	No duplicates (allows one NULL)
CHECK	Validates data against condition
DEFAULT	Fallback value
FOREIGN KEY	Links tables, `PRAGMA foreign_keys = ON`
CASE	Conditional expressions

📖 PART 1 — THEORY

Understanding JOINs

Why Do We Need JOINs?

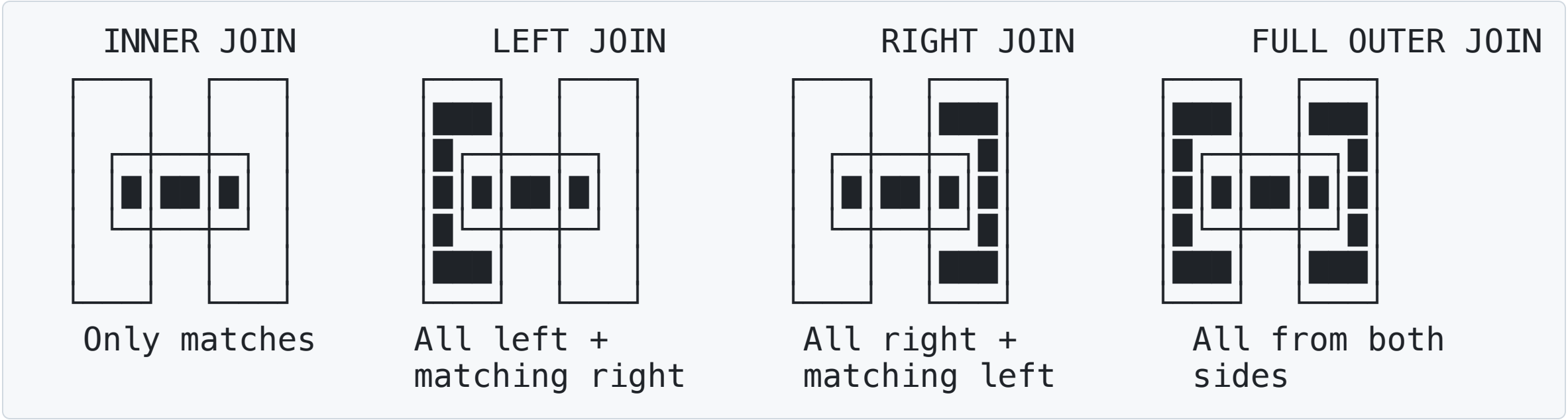
Problem: Data is spread across multiple tables

```
student                    department
1  Alice   1    ------>    1  Math Eng
2  Bob     2    ------>    2  Computer Eng
3  Carol   1    ------>    1  Math Eng
4  David NULL              3  Electrical
```

Goal: Combine data from related tables

- "Show me student names **with** their department names"
- Single-table query can't do this — we need JOINs!

Types of JOINs — Visual Overview



INNER JOIN

Returns only rows where there's a **match in both tables**.

```
SELECT s.first_name, s.last_name, d.dept_name
FROM   student s
INNER JOIN department d
    ON s.dept_id = d.dept_id;
```

first_name	last_name	dept_name
Alice	Yılmaz	Mathematical Engineering
Bob	Kaya	Computer Engineering
Carol	Demir	Mathematical Engineering

> ❌ David (dept_id = NULL) is excluded — no match in department
> ❌ Electrical Eng is excluded — no student in that department

INNER JOIN: How It Works Step by Step

```sql
-- Step 1: FROM – start with the left table (student)
-- Step 2: JOIN – combine with right table (department)
-- Step 3: ON – only keep rows where condition matches

SELECT s.first_name, d.dept_name
FROM   student s                        -- Left table
INNER JOIN department d                 -- Right table
    ON s.dept_id = d.dept_id;          -- Match condition
```

The ON clause:

- Specifies the **join condition**
- Usually matches a FK in one table to a PK in another
- Can use any comparison: `=`, `<`, `>`, `!=`, `BETWEEN`, etc.
- Multiple conditions: `ON a.id = b.id AND a.type = b.type`

LEFT JOIN (LEFT OUTER JOIN)

Non-matching right-side columns filled with **NULL**.

```sql
SELECT s.first_name, s.last_name, d.dept_name
FROM   student s
LEFT JOIN department d
    ON s.dept_id = d.dept_id;
```

first_name	last_name	dept_name
Alice	Yılmaz	Mathematical Engineering
Bob	Kaya	Computer Engineering
Carol	Demir	Mathematical Engineering
David	**Çelik**	**NULL**

> ✅ David appears even with no matching department
>
> 💡 LEFT JOIN is useful to find "students without a department"

Finding Rows With No Match (Anti-Join)

```sql
-- Students who have NO department assigned
SELECT s.first_name, s.last_name
FROM   student s
LEFT JOIN department d
    ON s.dept_id = d.dept_id
WHERE  d.dept_id IS NULL;      -- Only non-matching rows
```

first_name	last_name
David	Çelik

This pattern is called an Anti-Join:

1. LEFT JOIN to get all rows including non-matches
2. WHERE right-side PK IS NULL to keep only non-matches

> 💡 Very useful: "customers with no orders", "products never purchased"

RIGHT JOIN

```sql
-- In MySQL:
SELECT s.first_name, d.dept_name
FROM   student s
RIGHT JOIN department d
    ON s.dept_id = d.dept_id;
```

first_name	dept_name
Alice	Mathematical Engineering
Carol	Mathematical Engineering
Bob	Computer Engineering
NULL	**Electrical Engineering**

> ⚠️ **SQLite does NOT support RIGHT JOIN!**
>
> Workaround: Swap tables and use LEFT JOIN instead.

RIGHT JOIN Workaround in SQLite

```sql
-- MySQL RIGHT JOIN:
SELECT s.first_name, d.dept_name
FROM   student s
RIGHT JOIN department d ON s.dept_id = d.dept_id;

-- SQLite equivalent: Swap tables, use LEFT JOIN
SELECT s.first_name, d.dept_name
FROM   department d                    -- Now department is on the LEFT
LEFT JOIN student s                    -- Student is on the RIGHT
    ON d.dept_id = s.dept_id;      -- Same join condition

-- Result is identical!
```

> 💡 **Pro tip:** Most developers prefer LEFT JOIN over RIGHT JOIN.
> Just reorder your tables and always use LEFT JOIN.

FULL OUTER JOIN

```sql
-- MySQL (emulated with UNION):
SELECT s.first_name, d.dept_name
FROM student s LEFT JOIN department d ON s.dept_id = d.dept_id
UNION
SELECT s.first_name, d.dept_name
FROM department d LEFT JOIN student s ON d.dept_id = s.dept_id;
-- ⚠️ SQLite (3.39+) supports `FULL OUTER JOIN` directly!
```

first_name	dept_name
Alice	Mathematical Engineering
Bob	Computer Engineering
Carol	Mathematical Engineering
David	NULL
NULL	Electrical Engineering

🧠 Exam Focus: JOIN Types

> **Q:** Which type of JOIN returns all rows from the left table and matching rows from the right table, with NULLs for non-matches?
>
> A) INNER JOIN
> B) CROSS JOIN
> C) **LEFT JOIN** ✅
> D) FULL OUTER JOIN

Quick Reference:

JOIN Type	Returns
INNER	Only matching rows
RIGHT	All right + matching left
FULL OUTER	All from both

CROSS JOIN (Cartesian Product)

Returns **every combination** of rows from both tables.

```
SELECT s.first_name, c.course_name
FROM   student s
CROSS JOIN course c;
```

If student has 4 rows and course has 3 rows → result = **12 rows** (4 × 3)

When is CROSS JOIN useful?

```sql
-- Generate all possible schedules
SELECT t.time_slot, r.room_name
FROM time_slots t
CROSS JOIN rooms r;

-- Generate a calendar
SELECT m.month_name, d.day_num
FROM months m
CROSS JOIN (SELECT 1 AS day_num UNION SELECT 2 UNION SELECT 3) d;
```

> ⚠️ Be careful: CROSS JOIN can produce **huge** result sets!
>
> 1000 × 1000 = 1,000,000 rows

Self JOIN

```sql
-- Employee-Manager relationship
CREATE TABLE employee (
    emp_id      INTEGER PRIMARY KEY,
    name        TEXT,
    manager_id  INTEGER,
    FOREIGN KEY (manager_id) REFERENCES employee(emp_id)
);

INSERT INTO employee VALUES
    (1, 'Ali', NULL),       -- CEO (no manager)
    (2, 'Ayşe', 1),        -- Reports to Ali
    (3, 'Mehmet', 1),      -- Reports to Ali
    (4, 'Zeynep', 2),      -- Reports to Ayşe
    (5, 'Can', 2);         -- Reports to Ayşe

-- Self JOIN: Show employee with their manager's name
SELECT e.name AS employee,
       m.name AS manager
FROM   employee e
LEFT JOIN employee m ON e.manager_id = m.emp_id;
```

Self JOIN Result

employee	manager
Ali	NULL
Ayşe	Ali
Mehmet	Ali
Zeynep	Ayşe
Can	Ayşe

Key Points:

- Same table used **twice** with different aliases (`e` and `m`)
- LEFT JOIN includes employees with no manager (CEO)
- Common for: employee→manager, category→parent, node→parent

Multi-Table JOINs (3+ tables)

```sql
-- Student name + course name + grade (through enrollment)
SELECT s.first_name || ' ' || s.last_name AS student,
       c.course_name,
       e.grade
FROM   student s
JOIN   enrollment e ON s.student_id = e.student_id
JOIN   course c     ON e.course_id  = c.course_id;
```

How it works:

1. First JOIN: `student` ⋈ `enrollment` → intermediate result
2. Second JOIN: intermediate ⋈ `course` → final result

```
student ——— enrollment ——— course
   PK ←—— FK      FK —→ PK
```

> 💡 JOINs chain left to right. Each JOIN adds columns from a new table.

JOIN Best Practices

✅ Always use:

1. **Table aliases** — `FROM student s` (shorter, clearer)
2. **Explicit JOIN syntax** — `JOIN ... ON` (not comma-separated)
3. **Qualified column names** — `s.dept_id` (avoid ambiguity)

❌ Avoid:

```sql
-- Old-style implicit JOIN (comma syntax) - hard to read
SELECT s.first_name, d.dept_name
FROM student s, department d
WHERE s.dept_id = d.dept_id;

-- ✅ Modern explicit JOIN - clear and maintainable
SELECT s.first_name, d.dept_name
FROM student s
JOIN department d ON s.dept_id = d.dept_id;

-- The comma syntax still works but makes it easy to accidentally create a CROSS JOIN!
```

GROUP BY, Aggregates & HAVING

Aggregate Functions Review

Function	Description	Example
`COUNT(*)`	Count all rows	`SELECT COUNT(*) FROM student`
`COUNT(col)`	Count non-NULL values	`SELECT COUNT(email) FROM student`
`SUM(col)`	Sum of values	`SELECT SUM(budget) FROM dept`
`AVG(col)`	Average of values	`SELECT AVG(gpa) FROM student`
`MIN(col)`	Minimum value	`SELECT MIN(gpa) FROM student`
`MAX(col)`	Maximum value	`SELECT MAX(gpa) FROM student`
`GROUP_CONCAT(col)`	Concatenate values	`SELECT GROUP_CONCAT(name) FROM student`

```sql
SELECT COUNT(*) AS total,
       ROUND(AVG(gpa), 2) AS avg_gpa,
       MIN(gpa) AS min_gpa, MAX(gpa) AS max_gpa
FROM student;
```

🧠 Exam Focus: Aggregate Functions

> **Q:** Which aggregate function returns the total sum of values in a column?
>
> A) COUNT()
> B) MAX()
> C) AVG()
> D) **SUM()** ✅

Key Points:

- Aggregate functions ignore NULL values (except `COUNT(*)`)
- `COUNT(*)` counts all rows, `COUNT(column)` skips NULLs
- Cannot use aggregates in WHERE (use HAVING instead)

GROUP BY

`GROUP BY` **collapses rows** into groups, computing aggregates per group.

```sql
-- How many students in each department?
SELECT   dept_id,
         COUNT(*) AS student_count
FROM     student
GROUP BY dept_id;
-- Each unique `dept_id` becomes one output row.
```

dept_id	student_count
1	3
2	2
3	2
4	1
5	1
NULL	1

GROUP BY with Multiple Aggregates

```sql
-- Department statistics
SELECT   dept_id,
         COUNT(*) AS num_students,
         ROUND(AVG(gpa), 2) AS avg_gpa,
         MIN(gpa) AS min_gpa, MAX(gpa) AS max_gpa
FROM     student
GROUP BY dept_id
ORDER BY avg_gpa DESC;
```

Rules:

- Every column in SELECT must be:
  - In the GROUP BY clause, **OR**
  - Inside an aggregate function
- ❌ `SELECT first_name, dept_id, COUNT(*) FROM student GROUP BY dept_id`
  - Error! `first_name` is not grouped or aggregated

GROUP BY with JOINs

```sql
-- Count students per department (with department names)
SELECT   d.dept_name,
         COUNT(s.student_id) AS num_students,
         ROUND(AVG(s.gpa), 2) AS avg_gpa
FROM     department d
LEFT JOIN student s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY num_students DESC;
-- > LEFT JOIN ensures departments with 0 students still appear.
```

dept_name	num_students	avg_gpa
Mathematical Engineering	3	3.73
Computer Engineering	2	3.35
Electrical Engineering	2	2.90
Civil Engineering	1	3.65
Physics	1	3.10

HAVING — Filtering Groups

`HAVING` filters **after** grouping (while `WHERE` filters **before** grouping).

```sql
-- Departments with average GPA above 3.0
SELECT   dept_id,
         ROUND(AVG(gpa), 2) AS avg_gpa,
         COUNT(*) AS num_students
FROM     student
GROUP BY dept_id
HAVING   AVG(gpa) > 3.0
ORDER BY avg_gpa DESC;
```

WHERE vs HAVING:

Clause	Filters	When	Can Use Aggregates?
`WHERE`	Individual **rows**	**Before** GROUP BY	❌ No
`HAVING`	**Groups**	**After** GROUP BY	✅ Yes

🧠 Exam Focus: WHERE vs HAVING

> **Q:** What is the difference between WHERE and HAVING?
>
> A) WHERE filters columns; HAVING filters rows
>
> B) **WHERE filters rows before grouping; HAVING filters groups after GROUP BY** ✅
>
> C) HAVING is faster than WHERE
>
> D) WHERE can only be used with JOINs

```sql
-- Example: Both WHERE and HAVING in one query
SELECT   dept_id,
         AVG(gpa) AS avg_gpa
FROM     student
WHERE    birth_date >= '2002-01-01'    -- Filter rows FIRST
GROUP BY dept_id                        -- Then group
HAVING   AVG(gpa) > 3.0                -- Then filter groups
ORDER BY avg_gpa DESC;
```

GROUP_CONCAT — SQLite Special

```sql
-- List all student names per department
SELECT   dept_id,
         GROUP_CONCAT(first_name, ', ') AS students
FROM     student
GROUP BY dept_id;
```

dept_id	students
1	Alice, Carol, Irem
2	Bob, Elif
3	David, Gökhan
4	Fatma
5	Hakan

```sql
-- With ORDER BY inside (SQLite 3.44+)
SELECT dept_id,
```

☕ BREAK (15 minutes)

PART 2

Set Operations & Subqueries

UNION — Combining Result Sets

`UNION` stacks results **vertically** (combines rows from two queries).

```sql
-- All people (students and instructors)
SELECT first_name, last_name, 'Student' AS role
FROM   student
UNION
SELECT first_name, last_name, 'Instructor'
FROM   instructor;
```

Rules:

1. Both SELECTs must have the **same number of columns**
2. Corresponding columns must have **compatible types**
3. `UNION` removes duplicates (slow — needs sorting)
4. `UNION ALL` keeps all rows (fast — no dedup)

UNION vs UNION ALL

```sql
-- UNION: removes duplicates
SELECT city FROM customer
UNION
SELECT city FROM supplier;
-- If both tables have 'Istanbul', it appears ONCE

-- UNION ALL: keeps duplicates
SELECT city FROM customer
UNION ALL
SELECT city FROM supplier;
-- If both tables have 'Istanbul', it appears TWICE (or more)
```

When to use which?

Use	When
`UNION`	You need unique rows (default, slower)
`UNION ALL`	You want all rows, or know there are no duplicates (faster)

🧠 Exam Focus: Set Operations

> A) UNION ALL
> B) INTERSECT
> C) **UNION** ✅
> D) EXCEPT

Set Operations in SQLite:

Operator	Description	SQLite Support
`UNION`	All unique rows from both	✅ Yes
`UNION ALL`	All rows (with duplicates)	✅ Yes
`INTERSECT`	Only rows in **both** sets	✅ Yes
`EXCEPT`	Rows in first but **not** in second	✅ Yes

> 💡 MySQL does NOT support INTERSECT/EXCEPT. SQLite does!

INTERSECT and EXCEPT in SQLite

```sql
-- INTERSECT: Customers who are also students (by email)
SELECT email FROM customer
INTERSECT
SELECT email FROM student;

-- EXCEPT: Customers who are NOT students
SELECT email FROM customer
EXCEPT
SELECT email FROM student;

-- Practical example: Cities with both customers and suppliers
SELECT city FROM customer
INTERSECT
SELECT city FROM supplier;

-- Products ordered but not returned
SELECT product_id FROM order_item
EXCEPT
SELECT product_id FROM returns;
```

Subqueries — Queries Inside Queries

A subquery is a `SELECT` nested inside another SQL statement.

Types of Subqueries:

Type	Returns	Used In
Scalar	Single value	WHERE, SELECT
Row	Single row	WHERE
Table	Multiple rows/columns	FROM, JOIN
Correlated	Depends on outer query	WHERE, SELECT

```sql
-- Scalar subquery: students above average GPA
SELECT first_name, gpa
FROM   student
WHERE  gpa > (SELECT AVG(gpa) FROM student);
--              └─ returns single value: 3.35
```

Subquery in WHERE (IN, EXISTS)

```sql
-- IN subquery: Students enrolled in 'Applied SQL'
SELECT first_name, last_name
FROM   student
WHERE  student_id IN (
    SELECT student_id
    FROM   enrollment
    WHERE  course_id = (
        SELECT course_id FROM course WHERE course_name = 'Applied SQL'
    )
);

-- EXISTS subquery: Students who have at least one enrollment
SELECT first_name, last_name
FROM   student s
WHERE  EXISTS (
    SELECT 1
    FROM   enrollment e
    WHERE  e.student_id = s.student_id    -- Correlated!
);

-- NOT IN: Students with no enrollments
SELECT first_name, last_name
FROM   student
WHERE  student_id NOT IN (
    SELECT DISTINCT student_id FROM enrollment
);
```

Subquery in FROM (Derived Tables)

```sql
-- Average GPA per department, then filter
SELECT dept_id, avg_gpa
FROM (
    SELECT dept_id,
           AVG(gpa) AS avg_gpa
    FROM   student
    GROUP BY dept_id
) AS dept_stats                    -- Must have an alias!
WHERE  avg_gpa > 3.0;

-- Rank students by GPA and show rank
SELECT ranked.*
FROM (
    SELECT first_name,
           last_name,
           gpa,
           (SELECT COUNT(DISTINCT s2.gpa)
            FROM student s2
            WHERE s2.gpa > s1.gpa) + 1 AS rank
    FROM student s1
) AS ranked
ORDER BY rank;

-- 💡 Derived tables (subqueries in FROM) must always have an alias.
```

Subquery in SELECT (Scalar Subquery)

```sql
-- Show each student's GPA compared to department average
SELECT first_name, gpa, dept_id,
       (SELECT ROUND(AVG(s2.gpa), 2)
        FROM student s2
        WHERE s2.dept_id = s1.dept_id) AS dept_avg_gpa,
       ROUND(gpa - (SELECT AVG(s2.gpa)
                    FROM student s2
                    WHERE s2.dept_id = s1.dept_id), 2) AS diff_from_avg
FROM   student s1
ORDER BY dept_id, gpa DESC;
```

Correlated vs Non-Correlated:

- **Non-correlated**: subquery runs once, independently
  - `WHERE gpa > (SELECT AVG(gpa) FROM student)`
- **Correlated**: subquery references outer query, runs per row
  - `WHERE s2.dept_id = s1.dept_id` ← depends on outer `s1`

SQL Logical Execution Order 🔑

The order SQL **processes** clauses (NOT the order you write them):

```
1. FROM        — identify source tables
2. JOIN        — combine tables using ON condition
3. WHERE       — filter individual rows
4. GROUP BY    — group remaining rows
5. HAVING      — filter groups
6. SELECT      — compute output columns & expressions
7. DISTINCT    — remove duplicate rows
8. ORDER BY    — sort the result
9. LIMIT       — restrict number of returned rows
```

> 📝 **Write** order: `SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY → LIMIT`
>
> 📝 **Execute** order: `FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → DISTINCT → ORDER BY → LIMIT`

🧠 Exam Focus: SQL Execution Order

> **Q:** What is the correct logical order of SQL clause processing?
>
> A) SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY
>
> B) **FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY** ✅
>
> C) WHERE → FROM → SELECT → GROUP BY → ORDER BY
>
> D) FROM → SELECT → WHERE → ORDER BY → GROUP BY

Why does this matter?

```sql
SELECT dept_id, AVG(gpa) AS avg_gpa     -- Step 6
FROM   student                           -- Step 1
WHERE  birth_date >= '2002-01-01'        -- Step 3
GROUP BY dept_id                         -- Step 4
HAVING AVG(gpa) > 3.0                   -- Step 5 (can't use alias avg_gpa!)
ORDER BY avg_gpa DESC;                   -- Step 8 (CAN use alias here!)
```

Why You Can't Use Aliases in WHERE/HAVING

```sql
-- ❌ This FAILS (in standard SQL):
SELECT dept_id, AVG(gpa) AS avg_gpa
FROM student
GROUP BY dept_id
HAVING avg_gpa > 3.0;           -- avg_gpa not yet computed!

-- ✅ This WORKS:
SELECT dept_id, AVG(gpa) AS avg_gpa
FROM student
GROUP BY dept_id
HAVING AVG(gpa) > 3.0;         -- Repeat the expression

--
SELECT dept_id, AVG(gpa) AS avg_gpa
FROM student
GROUP BY dept_id
ORDER BY avg_gpa DESC;   -- ✅ ORDER BY CAN use aliases (processed after SELECT):
```

> 💡 SQLite is more lenient — it often allows aliases in HAVING. But for exams and portability, repeat the expression.

Complete Query Example

```sql
-- "Find departments with 2+ students born after 2001,
--  showing the department name, student count, and average GPA,
--  sorted by average GPA descending"

SELECT   d.dept_name,                                  -- 6. Compute columns
         COUNT(*) AS num_students,
         ROUND(AVG(s.gpa), 2) AS avg_gpa
FROM     student s                                      -- 1. Start with student
JOIN     department d ON s.dept_id = d.dept_id          -- 2. Join department
WHERE    s.birth_date > '2001-12-31'                    -- 3. Filter rows
GROUP BY d.dept_id, d.dept_name                         -- 4. Group by dept
HAVING   COUNT(*) >= 2                                  -- 5. Filter groups
ORDER BY avg_gpa DESC                                   -- 8. Sort
LIMIT    5;                                             -- 9. Limit results
```

PART 3 — CODING PRACTICE

Multi-Table Queries

💻 Practice: Setting Up the Database

We'll use the university database from Week 1 (with more data):

```
-- sqlite3 university_week3.db
PRAGMA foreign_keys = ON;
.headers on
.mode column

-- Tables: department, student, course, instructor, enrollment
-- (Use the setup script from week01_practice.sql)
-- Or load it: .read ../code/week01_practice.sql
```

> 💡 Make sure you have the Week 1 database loaded with data!

💻 Practice: INNER JOIN Exercises

```sql
-- 1. Students with their department names
SELECT s.first_name, s.last_name, d.dept_name
FROM   student s
INNER JOIN department d ON s.dept_id = d.dept_id;

-- 2. Course enrollments with student and course names
SELECT s.first_name || ' ' || s.last_name AS student,
       c.course_name,
       e.grade,
       e.semester
FROM   enrollment e
JOIN   student s ON e.student_id = s.student_id
JOIN   course c  ON e.course_id  = c.course_id
ORDER BY s.last_name, c.course_name;

-- 3. Students and courses in the same department
SELECT s.first_name, c.course_name, d.dept_name
FROM   student s
JOIN   department d ON s.dept_id = d.dept_id
JOIN   course c     ON c.dept_id = d.dept_id;
```

💻 Practice: LEFT JOIN Exercises

```sql
-- 4. All students, including those with no department
SELECT s.first_name, s.last_name,
       COALESCE(d.dept_name, 'Unassigned') AS department
FROM   student s
LEFT JOIN department d ON s.dept_id = d.dept_id;

-- 5. All departments with student count (including empty depts)
SELECT d.dept_name,
       COUNT(s.student_id) AS num_students
FROM   department d
LEFT JOIN student s ON d.dept_id = s.dept_id
GROUP BY d.dept_id, d.dept_name
ORDER BY num_students DESC;

-- 6. Students who are NOT enrolled in any course
SELECT s.first_name, s.last_name
FROM   student s
LEFT JOIN enrollment e ON s.student_id = e.student_id
WHERE  e.enrollment_id IS NULL;
```

💻 Practice: Multi-Table JOINs

```sql
-- 7. Complete enrollment report
SELECT s.first_name || ' ' || s.last_name AS student,
       d.dept_name AS department,
       c.course_name AS course,
       c.credits,
       e.grade,
       e.semester
FROM   enrollment e
JOIN   student s    ON e.student_id = s.student_id
JOIN   course c     ON e.course_id  = c.course_id
JOIN   department d ON s.dept_id    = d.dept_id
ORDER BY d.dept_name, s.last_name;

-- 8. Courses with no enrollments
SELECT c.course_name, c.credits
FROM   course c
LEFT JOIN enrollment e ON c.course_id = e.course_id
WHERE  e.enrollment_id IS NULL;
```

Practice: Self JOIN

```sql
-- 9. Create and populate employee table
CREATE TABLE IF NOT EXISTS employee (
    emp_id      INTEGER PRIMARY KEY,
    name        TEXT NOT NULL,
    position    TEXT,
    manager_id  INTEGER,
    salary      REAL,
    FOREIGN KEY (manager_id) REFERENCES employee(emp_id)
);

DELETE FROM employee;
INSERT INTO employee VALUES
    (1, 'Ali Yılmaz',    'CEO',           NULL, 150000),
    (2, 'Ayşe Kara',     'VP Engineering', 1,   120000),
    (3, 'Mehmet Demir',  'VP Sales',       1,   110000),
    (4, 'Zeynep Çelik',  'Sr. Developer', 2,    90000),
    (5, 'Can Öz',        'Developer',      2,    75000),
    (6, 'Deniz Ak',      'Sales Manager',  3,    80000),
    (7, 'Ece Tan',       'Developer',      4,    70000);

-- Employee with manager name
SELECT e.name AS employee,
       e.position,
       COALESCE(m.name, '(none)') AS manager
FROM   employee e
LEFT JOIN employee m ON e.manager_id = m.emp_id;
```

💻 Practice: Self JOIN (continued)

```sql
-- 10. Employees who earn more than their manager
SELECT e.name AS employee,
       e.salary AS emp_salary,
       m.name AS manager,
       m.salary AS mgr_salary
FROM   employee e
JOIN   employee m ON e.manager_id = m.emp_id
WHERE  e.salary > m.salary;

-- 11. Count direct reports per manager
SELECT m.name AS manager,
       COUNT(e.emp_id) AS direct_reports
FROM   employee m
LEFT JOIN employee e ON e.manager_id = m.emp_id
GROUP BY m.emp_id, m.name
HAVING COUNT(e.emp_id) > 0
ORDER BY direct_reports DESC;
```

💻 Practice: GROUP BY + HAVING

```sql
-- 12. Number of enrollments per course
SELECT c.course_name,
       COUNT(*) AS enrollment_count
FROM   enrollment e
JOIN   course c ON e.course_id = c.course_id
GROUP BY c.course_id, c.course_name
ORDER BY enrollment_count DESC;

-- 13. Courses with more than 2 students
SELECT c.course_name,
       COUNT(*) AS num_students
FROM   enrollment e
JOIN   course c ON e.course_id = c.course_id
GROUP BY c.course_id
HAVING COUNT(*) > 2;

-- 14. Grade distribution across all courses
SELECT grade,
       COUNT(*) AS count
FROM   enrollment
WHERE  grade IS NOT NULL
GROUP BY grade
ORDER BY grade;
```

💻 Practice: GROUP BY + HAVING (continued)

```sql
-- 15. Average GPA per department (only depts with avg > 3.0)
SELECT d.dept_name,
       COUNT(*) AS students,
       ROUND(AVG(s.gpa), 2) AS avg_gpa
FROM   student s
JOIN   department d ON s.dept_id = d.dept_id
GROUP BY d.dept_id
HAVING AVG(s.gpa) > 3.0
ORDER BY avg_gpa DESC;

-- 16. Students who enrolled in more than 1 course
SELECT s.first_name, s.last_name,
       COUNT(*) AS courses_enrolled
FROM   student s
JOIN   enrollment e ON s.student_id = e.student_id
GROUP BY s.student_id
HAVING COUNT(*) > 1
ORDER BY courses_enrolled DESC;

-- 17. Total credits per student
SELECT s.first_name, s.last_name,
       SUM(c.credits) AS total_credits
FROM   student s
JOIN   enrollment e ON s.student_id = e.student_id
JOIN   course c     ON e.course_id  = c.course_id
GROUP BY s.student_id
ORDER BY total_credits DESC;
```

💻 Practice: UNION & Set Operations

```sql
-- 18. All people in the university (students + instructors)
SELECT first_name, last_name, 'Student' AS role
FROM   student
UNION
SELECT first_name, last_name, 'Instructor'
FROM   instructor
ORDER BY role, last_name;

-- 19. UNION ALL — with duplicates
SELECT first_name FROM student
UNION ALL
SELECT first_name FROM instructor;

-- 20. INTERSECT — departments that have BOTH students AND courses
SELECT dept_id FROM student
INTERSECT
SELECT dept_id FROM course;

-- 21. EXCEPT — departments with students but NO courses
SELECT dept_id FROM student WHERE dept_id IS NOT NULL
EXCEPT
SELECT dept_id FROM course WHERE dept_id IS NOT NULL;
```

Practice: Subqueries

```sql
-- 22. Students above average GPA
SELECT first_name, last_name, gpa
FROM   student
WHERE  gpa > (SELECT AVG(gpa) FROM student)
ORDER BY gpa DESC;

-- 23. Students in the department with highest budget
SELECT first_name, last_name
FROM   student
WHERE  dept_id = (
    SELECT dept_id FROM department ORDER BY budget DESC LIMIT 1
);

-- 24. Courses taken by the student with highest GPA
SELECT c.course_name
FROM   course c
WHERE  c.course_id IN (
    SELECT e.course_id
    FROM   enrollment e
    WHERE  e.student_id = (
        SELECT student_id FROM student ORDER BY gpa DESC LIMIT 1
    )
);
```

💻 Practice: Subqueries (continued)

```sql
-- 25. Students with GPA above their department average
SELECT s.first_name, s.last_name, s.gpa, s.dept_id
FROM   student s
WHERE  s.gpa > (
    SELECT AVG(s2.gpa)
    FROM   student s2
    WHERE  s2.dept_id = s.dept_id    -- Correlated subquery
);

-- 26. Departments with below-average budget
SELECT dept_name, budget
FROM   department
WHERE  budget < (SELECT AVG(budget) FROM department);

-- 27. Students enrolled in 'Applied SQL' (nested subquery)
SELECT first_name, last_name
FROM   student
WHERE  student_id IN (
    SELECT student_id FROM enrollment
    WHERE course_id IN (
        SELECT course_id FROM course WHERE course_name = 'Applied SQL'
    )
);
```

💻 Practice: Derived Tables & Complex Queries

```sql
-- 28. Rank students by GPA (without window functions)
SELECT s.first_name, s.last_name, s.gpa,
       (SELECT COUNT(DISTINCT s2.gpa)
        FROM student s2
        WHERE s2.gpa > s.gpa) + 1 AS rank
FROM   student s
ORDER BY rank;

-- 29. Department with most students (using derived table)
SELECT dept_name, student_count
FROM (
    SELECT d.dept_name,
           COUNT(s.student_id) AS student_count
    FROM department d
    LEFT JOIN student s ON d.dept_id = s.dept_id
    GROUP BY d.dept_id, d.dept_name
) AS dept_counts
ORDER BY student_count DESC
LIMIT 1;

-- 30. Students who got an 'A' in every course they took
SELECT first_name, last_name
FROM student s
WHERE NOT EXISTS (
    SELECT 1 FROM enrollment e
    WHERE e.student_id = s.student_id AND e.grade != 'A'
)
AND EXISTS (
    SELECT 1 FROM enrollment e
    WHERE e.student_id = s.student_id
);
```

💻 Practice: E-Commerce Multi-Table Queries

```sql
-- Load the e-commerce database from Week 2
-- sqlite3 ecommerce.db

-- 31. Orders with customer names and item details
SELECT c.first_name || ' ' || c.last_name AS customer,
       o.order_id,
       o.order_date,
       p.product_name,
       oi.quantity,
       PRINTF('$%.2f', oi.unit_price * oi.quantity / 100.0) AS line_total
FROM   orders o
JOIN   customer c   ON o.customer_id = c.customer_id
JOIN   order_item oi ON o.order_id    = oi.order_id
JOIN   product p    ON oi.product_id  = p.product_id
ORDER BY o.order_date, o.order_id;
```

💻 Practice: E-Commerce Analytics

```sql
-- 32. Revenue per product category
SELECT cat.category_name,
       COUNT(oi.item_id) AS items_sold,
       PRINTF('$%.2f', SUM(oi.quantity * oi.unit_price) / 100.0) AS revenue
FROM   order_item oi
JOIN   product p  ON oi.product_id = p.product_id
JOIN   category cat ON p.category_id = cat.category_id
GROUP BY cat.category_id
ORDER BY SUM(oi.quantity * oi.unit_price) DESC;

-- 33. Customers who spent more than $100
SELECT c.first_name, c.last_name,
       PRINTF('$%.2f', SUM(o.total_cents) / 100.0) AS total_spent
FROM   customer c
JOIN   orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id
HAVING SUM(o.total_cents) > 10000    -- > $100 in cents
ORDER BY total_spent DESC;

-- 34. Products never ordered
SELECT p.product_name
FROM   product p
LEFT JOIN order_item oi ON p.product_id = oi.product_id
WHERE  oi.item_id IS NULL;
```

Practice: Complex Analytics

```sql
-- 35. Most popular product (most ordered)
SELECT p.product_name,
       SUM(oi.quantity) AS total_qty_sold
FROM   order_item oi
JOIN   product p ON oi.product_id = p.product_id
GROUP BY p.product_id
ORDER BY total_qty_sold DESC
LIMIT 1;

-- 36. Average order value
SELECT PRINTF('$%.2f', AVG(total_cents) / 100.0) AS avg_order_value
FROM   orders
WHERE  status != 'cancelled';

-- 37. Customer order frequency
SELECT c.first_name || ' ' || c.last_name AS customer,
       COUNT(o.order_id) AS order_count,
       PRINTF('$%.2f', AVG(o.total_cents) / 100.0) AS avg_order
FROM   customer c
LEFT JOIN orders o ON c.customer_id = o.customer_id
GROUP BY c.customer_id
ORDER BY order_count DESC;
```

Summary: JOINs Cheat Sheet

JOIN Type	SQLite Support	Returns
`INNER JOIN`	✅	Only matching rows
`LEFT JOIN`	✅	All left + matching right
`RIGHT JOIN`	❌	Swap tables → use LEFT JOIN
`FULL OUTER JOIN`	✅ (3.39+)	All rows from both
`CROSS JOIN`	✅	Cartesian product
`Self JOIN`	✅	Table joined to itself

Key Exam Points:

- `ON` defines the join condition
- LEFT JOIN + `WHERE right.pk IS NULL` = Anti-join
- Execution order: `FROM → JOIN → WHERE → GROUP BY → HAVING → SELECT → ORDER BY`

Summary: Aggregates & Subqueries

GROUP BY + HAVING:

- GROUP BY collapses rows into groups
- Aggregate functions compute per group (COUNT, SUM, AVG, MIN, MAX)
- WHERE filters rows **before** grouping
- HAVING filters groups **after** grouping

Subqueries:

- Scalar → single value (in WHERE, SELECT)
- Table → derived table (in FROM, requires alias)
- Correlated → references outer query (runs per row)
- IN, NOT IN, EXISTS, NOT EXISTS

📝 Practice Exercises (Homework)

1. **Using the university database:**
   - Find the top student in each department (highest GPA)
   - Show the grade distribution for each course
   - Find students enrolled in more than 2 courses
2. **Using the e-commerce database:**
   - Find customers who ordered products from every category
   - Calculate the total revenue per month
   - List products that are in stock but have never been ordered
3. **Challenge:** Write a query that shows a "report card" for each student — listing all courses with grades and the student's overall GPA.

🔮 Next Week Preview

Week 4: Indexes & Performance

- What are indexes and why they matter
- Creating and managing indexes
- B-Tree index structure
- EXPLAIN and query analysis
- Index best practices
- Performance optimization tips

Preparation:

- 📖 Read: *Learning SQL* Ch. 13 (Indexes)
- 🎓 Watch: CS50 SQL Week 4 — Optimizing

Thank You!

Questions?

Next class: Indexes & Performance Optimization